

Structured Implementation Prompt

Copy the prompt below into a later coding session. Adjust the chosen stack only before implementation begins.

You are implementing the Ottodot Full-Stack Engineer take-home. Work within a strict four-hour total timebox and build the smallest reliable trial-booking slice. Do not implement regular enrollment or unrelated product features.

Follow compact spec-driven development. Treat the approved requirements, acceptance criteria, technical decisions, API contracts, and ordered tasks as the baseline. If implementation requires a behavioral change, update the corresponding specification and test rather than silently diverging.

Goal

Create a runnable TypeScript application in a public-repository-ready state that lets a seeded parent select a child and trial class, create a booking, submit a mock payment result, view booking status, and view a confirmed-only class roster.

Preferred stack

- Next.js App Router with a minimal accessible UI and route handlers
- PostgreSQL using Docker Compose
- Drizzle ORM, with explicit SQL for partial indexes and row locks where clearer
- Vitest for tests

If the existing repository already establishes an equivalent stack, preserve it. Do not replace working foundations without a concrete reason.

Use a PostgreSQL transaction and an explicit state machine for the implemented workflow. Do not implement Kafka or a distributed Saga. Document real payment compensation, a transactional outbox, and the conditions that could justify Kafka under production evolution. Use one short-lived feature branch rather than full GitFlow.

Required invariants

1. Confirmed bookings for a class never exceed its stored capacity; seeded capacity is 4.
2. A student never has more than one active (`pending_payment` or `confirmed`) booking for the same class.
3. Only a booking with a recorded successful payment result can become confirmed.
4. Payment failure never places a student on the confirmed roster.
5. Roster queries return confirmed bookings only.
6. Replayed booking and payment commands are idempotent.

Required statuses

Use `pending_payment`, `confirmed`, `payment_failed`, `capacity_unavailable`, and `cancelled`, or clearly document an equivalent vocabulary.

Required data model

Create small tables for parents, students, trial classes, bookings, and payment attempts. Add timestamps and foreign keys. Add:

- a unique booking-creation idempotency key;
- a unique payment-attempt idempotency key;
- a partial unique index on `(student_id, trial_class_id)` for active booking statuses;
- a positive capacity check on trial classes.

Required API behavior

Implement equivalent endpoints/functions for:

- listing a parent's children;
- listing trial classes with indicative availability;
- creating a pending booking;
- recording a mock success/failure payment result;
- fetching booking status;
- fetching a confirmed-only class roster.

Return stable domain error codes and clear HTTP statuses. Treat UI availability as advisory.

Concurrency requirement

For a successful payment result, perform the decisive work in one database transaction:

1. Lock and validate the booking.
2. handle an already-seen idempotency key by returning its stored outcome;
3. lock the relevant trial class row;
4. record the payment attempt;
5. count confirmed bookings while the class lock is held;
6. confirm only if the count is below capacity;
7. otherwise mark `capacity_unavailable`;
8. commit.

Use one consistent lock order. Do not call an external service while holding the transaction open. In the mock flow, retain successful payment audit data if capacity is lost and explain that production would authorize before seat claim and capture afterward, or void/refund.

Seed data

Include:

- at least one class with ordinary availability;
- one class with exactly three confirmed students;
- data or fixtures that demonstrate duplicate booking;
- a payment failure case.

Use synthetic data only.

Tests

At minimum, automate:

- happy path;
- payment failure with unchanged roster;
- duplicate active/confirmed booking prevention;
- full class rejection;
- idempotent replay;
- roster filtering;
- deterministic last-seat race using two concurrent connections, asserting exactly one winner and a final confirmed count of four.

Do not mock away the database in the concurrency test.

Documentation

Create or update:

- `README.md` : setup, run, test, what was built, time spent, assumptions, schema/API, statuses, last-seat approach and tradeoffs, layer responsibilities, deliberate cuts, monitoring, and next steps.
- `AI_USAGE.md` : tools used, uses, one acceleration, one rejected/corrected suggestion, workflow improvements, and verification.

Include a 5-8 minute demo outline. Mention that zip files are not accepted and the final deliverable must be a public GitHub repository plus a video link.

Scope limits

Do not add real payments, regular enrollment, full authentication, notifications, complex admin screens, broad design-system work, or deployment infrastructure. A clean minimal UI is sufficient.

Working method

1. Inspect the repository and preserve unrelated user changes.
2. Confirm the approved specification, plan, contracts, tasks, and acceptance-to-test mapping.
3. Create or use the short-lived `feature/trial-booking` branch.

4. Implement schema and invariants first.
5. Implement the transaction and its integration tests.
6. Add the thinnest API and UI needed to demonstrate the flow.
7. Run migrations, seed, type checks, lint, and tests.
8. Verify the race repeatedly.
9. Reconcile specifications and documentation with actual behavior.
10. Stop at the timebox and document remaining work instead of hiding it.

Report the files changed, commands run, test results, tradeoffs, and any unfinished item. Do not claim a behavior is verified unless a corresponding test or manual check was run.
